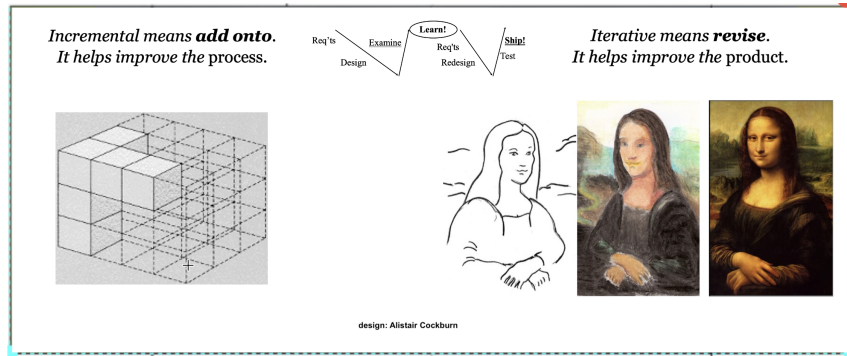


The Incremental and the Iterative

Two approaches to software development. What's the difference? Why does it matter?



To Quote Alistair Cockburn
(well, his mug)

Incremental means add on to. It helps improve the process. Iterative means revise. It helps improve the product.

Iterative Development

Supporting the Product



Deliver a *whole, working version* each time.

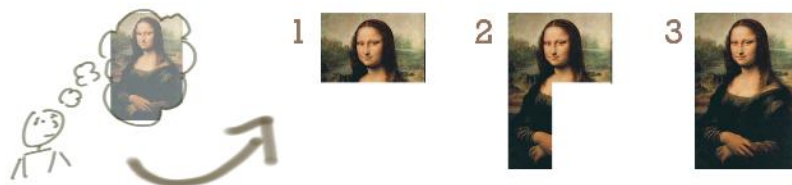
You can learn from users. Discover you've already "won". Change direction.

The feedback loop matters more than the plan.

Credit: Jeff Patton

Incremental Development

Supporting the Process



Deliver *pieces*, assemble at the end.

Break big problems into chunks. Manage complexity.

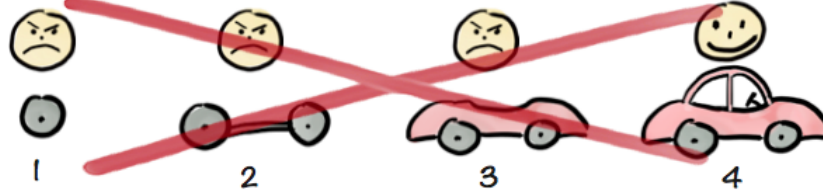
But pieces aren't usable until assembled.

Credit: Jeff Patton

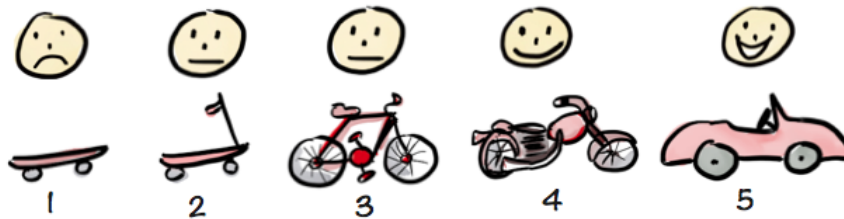
The Ideal

Kniberg's Skateboard → Car

Not like this....



Like this!



Bottom row: Skateboard → scooter → bike → car

Each iteration delivers *transportation*.

This is **external product iteration** - the ideal.

Credit: [Henrik Kniberg](#)

The First Danger

Being Purely Incremental

Building pieces perfectly but never validating the whole solves the user's problem.

You can execute flawlessly on the **wrong product**.

The Second Danger

Refusing to Increment

"Everything must be a complete, releasable feature!"

But if that takes two months:

- Large batches
- Invisible work
- No internal feedback
- Integration hell

Massive, unwieldy increments.

The "Real" World

Iteration Is Often Constrained



You can't always ship to users incrementally:

- Regulations
- Market constraints
- Technical limitations

But you don't give up on iteration entirely.

What sort of idiot builds a car like this?



yes it's steel thread time



The Steel Thread

Multiple Dimensions Simultaneously



Now, from the point of view of that end user, who wanted a car, the steel thread is incremental.

But from another view, it's **iterative in multiple ways**.

Steel Thread: Integration

Always Present, Always Improving

Integration is iterated from day one.

Not: Build pieces → assemble → hope

But: Integrated flow → refine connections → validate continuously

Stub becomes real client. Mock becomes live service.

The system is always wired together.

Steel Thread: Components

Iteratively Refined

Components are **added** incrementally.

But each **evolves** iteratively:

Stub → Partial → Full implementation

Incrementing *what exists* while iterating *what it does*.

And as your steel thread will include "stubs" at all the integration points, you're set up for iteration from the start.

Steel Thread: Behaviour

Present From Iteration One

The behavior exists from the first test.

No step change from "no feature" to "feature exists."

Even the enabling test describes the actual behavior you want.

Just with constrained scope or audience initially.

Steel Thread: Product Feedback

Iterate With Constrained Audiences

and because these are iterations, they can be show to users. Well, maybe not all the users. Maybe just some. Maybe just *you*. But it's still a user, and they can still give feedback.

Show it to:

- The team
- The PO
- A group of friendly users
- Your mum
- Yourself

Get feedback on the whole feature from the beginning.

Internal product iteration, not just integration testing.

What Actually Increments?

Only incremental from a certain point of view

Some behaviour is there from iteration one.

What increments:

- Production-readiness
- Robustness (stub → real)
- Scale/performance
- Audience reach (team → all users)

Steel thread: iterate until you hit some tipping point, ready for full release.

The Key Insight

Iterative (development)

Within Incremental (product)

Cockburn: Incremental delivery improves the process by breaking work into pieces.

We agree - with a caveat:

Even incremental features can be delivered iteratively:

- At the system level (integration)
 - At the product level (constrained audiences)
-

Features Are Increments

How You Build Them Is Iterative

Adding a new feature? That's incrementing functionality.

But build it iteratively:

- Iterate its integration with existing system
- Iterate its behaviour with friendly users
- Keep learning

The feature is an increment. The approach is iterative.

The Practical Application

(or what does this mean for stories?)

1. Small is Beautiful

keep it that way

keep your stories small to benefit the product

Stories that you're working on should be *small*. The benefits of working in small batches are huge.

Small keeps your feedback loop tight, keeps your focus tight, keeps your work visible, keeps you on track.

About two days. Fight me,

Make the process visible.

2. Ideally, Product Iterations

iterating on the product features

And ideally, each story should be an iteration of a product. At the end of a story you immediately release value which the user can give you feedback on.

3. Otherwise, Steel Thread

aka iterating without release

Minimum: Iterate Integration, but at maximum you are building tiny little bits of product that can be consumed by the right audience *for feedback* so you can *learn*. You, the PO, your close family.

When you can't iterate externally:

1. Keep integration present from day one
 2. Increment components while iterating connections
 3. Get product feedback from constrained audiences
-

4. Purely Incremental as the Last Resort

i.e. multiple teams, parallelism

Remember

Work Gets Broken Down Anyway

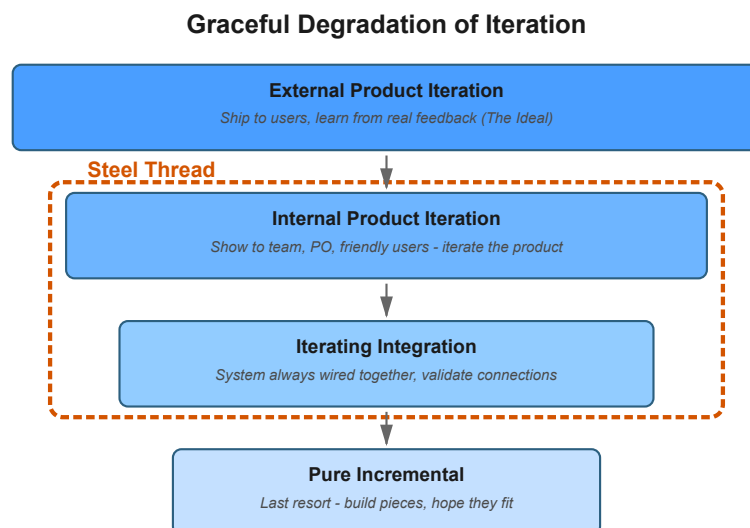
Humans are sequential. We work one thing at a time.

Work will be decomposed.

The question is: **by design or by accident?**

In Summary

Iterate to Learn Increment to Manage



Steel thread lets you do both.

Integration iterates from day one. Components increment while improving. Behaviour validates with constrained users.

Don't let "complete stories" hide months where nothing integrates.

BUT WAIT

What's really going on here

I said iterate to learn, but really I also mean, iterate to reduce risks, to learn to improve, but to avoid dangers.

The danger isn't "not being iterative" - iteration isn't a goal in itself. And "small" isn't an end in itself.

The benefits of these two

So really what I mean is

In a system governed by ignorance (i.e. software)

**Continuity
distributes surprise
Discontinuity
concentrates surprise**

both at the product and technical level

both nasty and pleasant

Questions?

<https://www.se.rit.edu/~swen-256/resources/UsingBothIncrementalandIterativeDevelopment-AlistairCockburn.pdf>